自然語言處理與應用
Natural Language Processing and Applications

文字向量基礎

Instructor: 林英嘉 (Ying-Jia Lin)
2026/03/02

GitHub

Slido
NLP0302
(Link)

Outline

- 文字向量基礎
  - **Introduction to embeddings**
  - Sentence embeddings
  - Word embeddings

影像 vs. 文字

⟶ 28 x 28 的矩陣

I want to study AI ⟶ ?

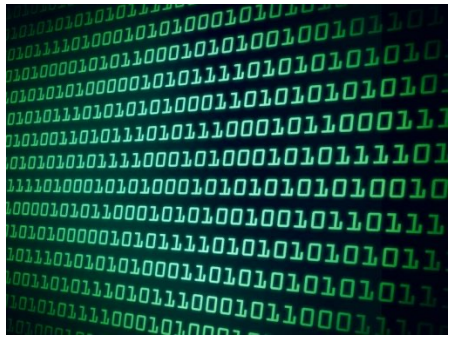
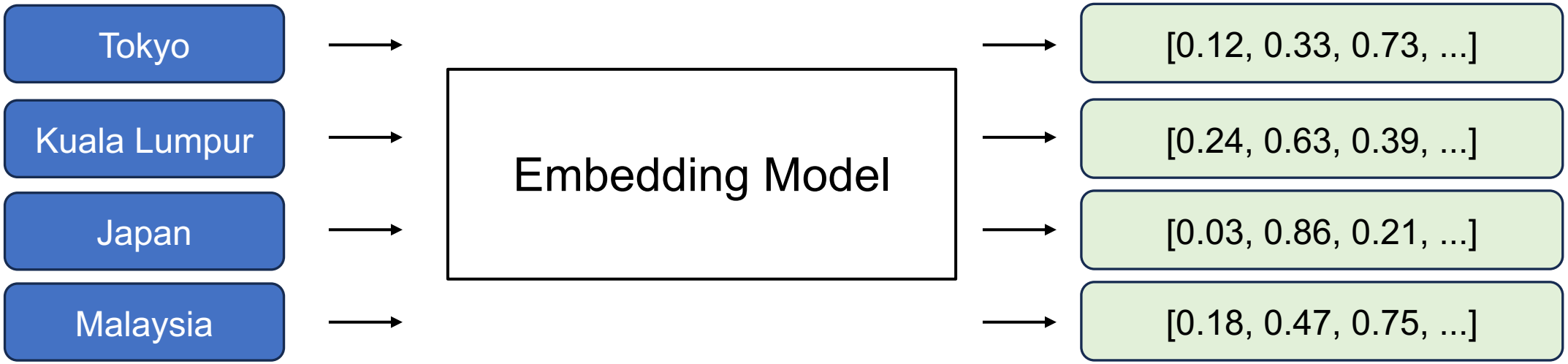


Figure source: https://kopu.chat/wp-content/uploads/2017/04/binary-507x380.jpeg

Embeddings（嵌入）

💡在NLP 中，Embeddings 代表把文字（word / token / sentence）映射成向量，使得向量之間的距離或方向能反映它在語料中的用法與語意相似性。



Embedding: 把(語義)資訊嵌入到向量中

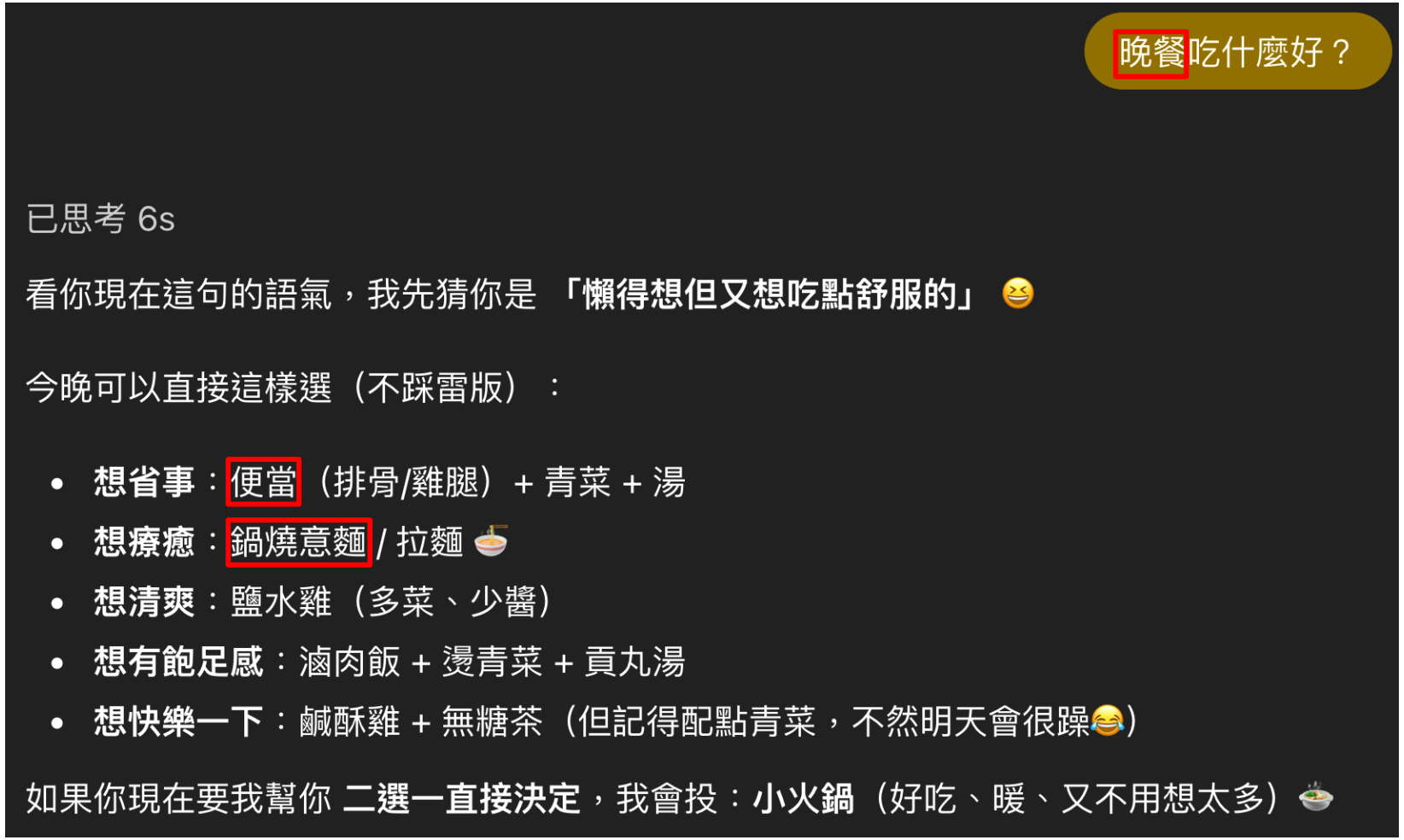
Tokyo	[0.12, 0.33, 0.73, ...]
Kuala Lumpur	[0.24, 0.63, 0.39, ...]
Japan	[0.03, 0.86, 0.21, ...]
Malaysia	[0.18, 0.47, 0.75, ...]

詞向量視覺化

- **Word embedding model**: glove-wiki-gigaword-100
- **Dimension reduction**: t-SNE
- **Dataset**: Mikolov et al., 2013

Try it:
https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/

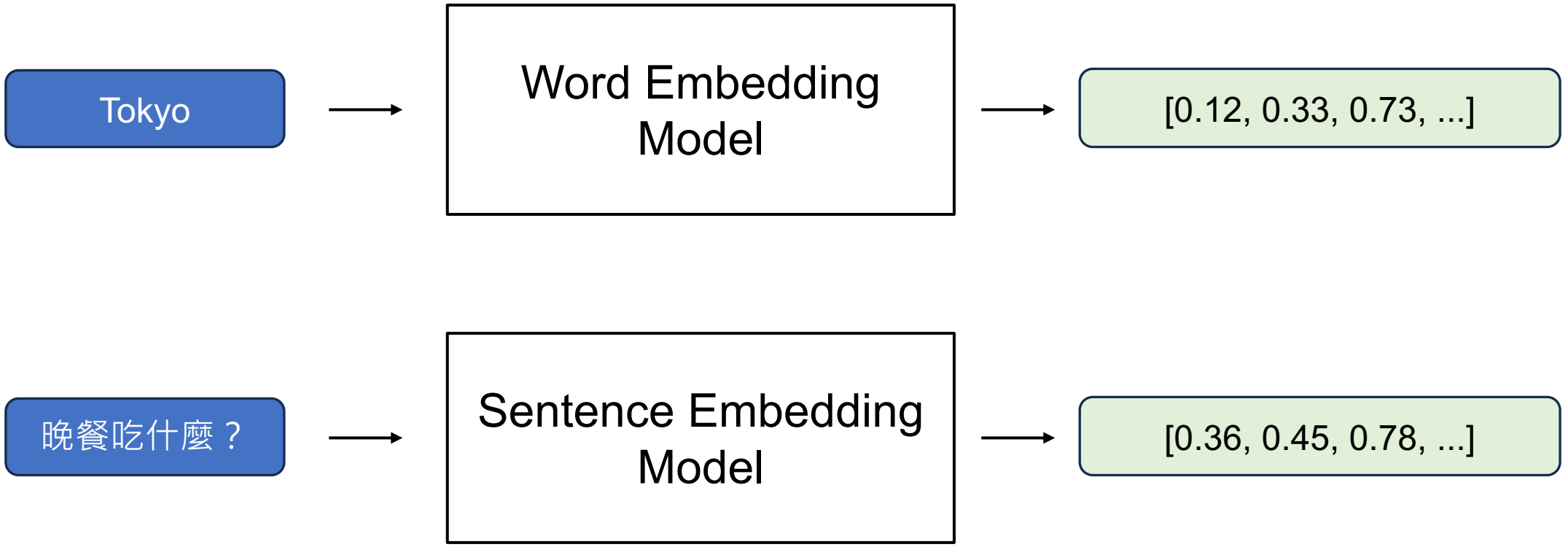
ChatGPT 也是在根據向量的語義進行回答



Outline

- 文字向量基礎
  - **Introduction to embeddings**
  - Sentence embeddings
  - Word embeddings

詞向量 vs. 句向量


Tokyo
Word Embedding Model
[0.12, 0.33, 0.73, ...]
晚餐吃什麼？
Sentence Embedding Model
[0.36, 0.45, 0.78, ...]

Google Search (Retrieval)

- Retrieval: get relevant information from a pool (like a search engine)

Google Search vs. 向量化

NLP 的 Embeddings: 語意資訊被嵌入到（高維度）向量空間

Bag-of-words Model

Meaning: Each document (sentence) carries a bag of words.

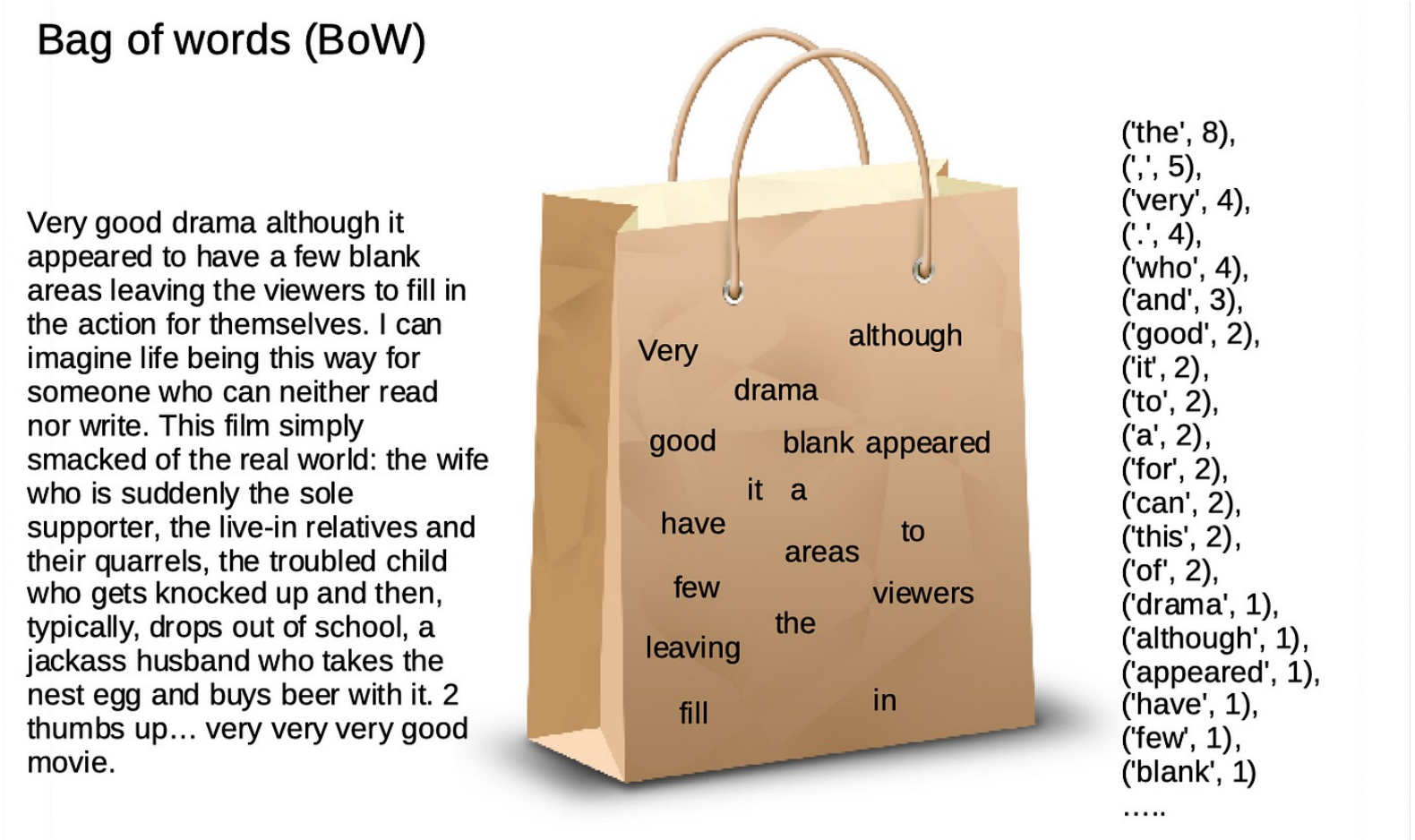
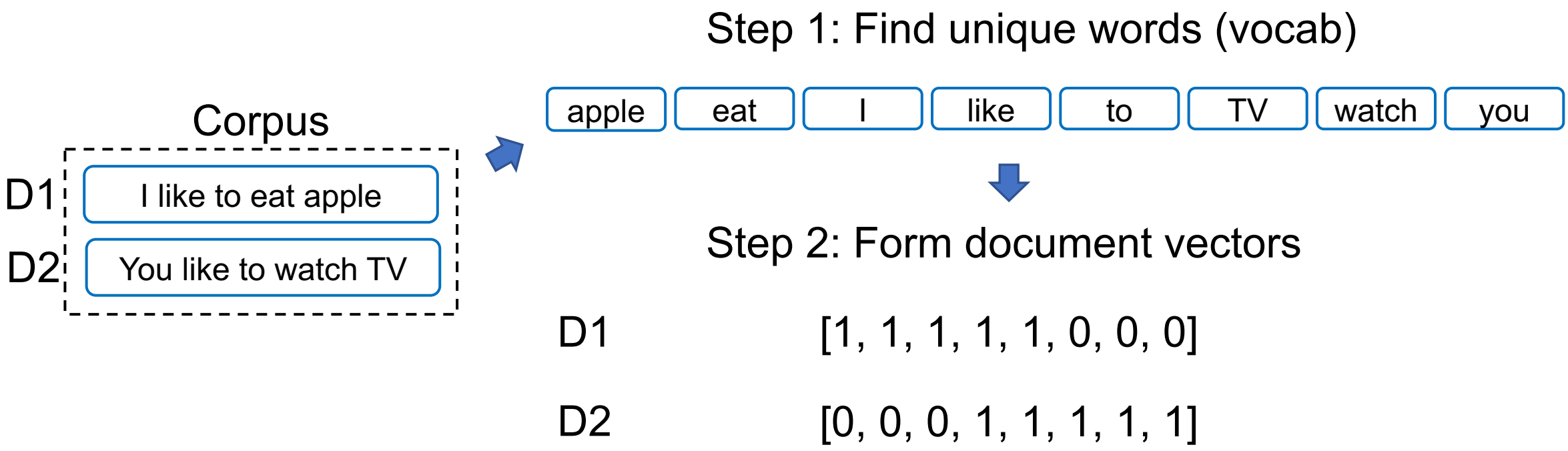


Figure source: https://sfhsu29.medium.com/nlp-入門-1-text-classification-sentiment-analysis-極簡易情感分類器-bag-of-words-naive-bayes-e40d61de9a7f

Bag-of-words Model (Example 1)

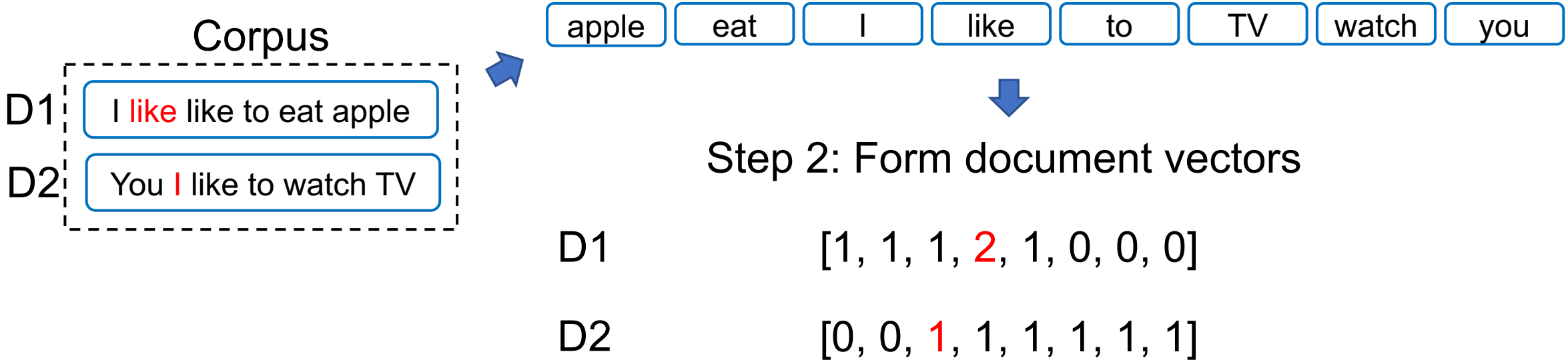
https://github.com/mcps5601/CGUNLP_2025_Spring/code/bow.py



Notes:

- The vector size is equal to the vocab size.
- Each position of a vector is corresponding to the position in the vocab.

Bag-of-words Model (Example 2)



Notes:

- The vector size is equal to the vocab size.
- Each position of a vector is corresponding to the position in the vocab.

TF-IDF

- TF (Term Frequency; 詞頻)
- IDF (Inverse Document Frequency; 逆文本頻率)

TF x IDF = 關鍵字分數

某詞出現在該句子的頻率

某詞出現在全部文本的頻率的倒數

TF-IDF 為什麼這樣設計？

想像你撿到了班上三位同學的日記：

日記 A (小明的)：「我 今天 去 吃 **漢堡**，**漢堡** 超 好吃。」

日記 B (小華的)：「我 今天 去 學校 上課，好 累。」

日記 C (小美的)：「我 今天 跟 媽媽 去 買 菜。」

日記 A 裡的內容觀察：

單字	TF (在這篇很多?)	IDF (在別篇很少?)	結果 (TF-IDF)	意義
我	是 (1次)	否 (每句都有)	**低分**	雜訊/不重要
今天	是 (1次)	否 (每句都有)	**低分**	雜訊/不重要
漢堡	**是 (2次)**	**是 (別句都沒提)**	**高分**	**關鍵字！**

TF-IDF 的公式

- The mathematical representation of TF-IDF:

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

where

$$\text{TF}_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}} \qquad \text{IDF}_\text{i} = \log \frac{|D|}{|\{j : t_i \in d_j\}|}$$

  - Where $n_{i,j}$ is the i-th word in the j-th document in the dataset.
- TF (Term Frequency)
  - Represents the "frequency" of a term appearing in a text.
- IDF (Inverse Document Frequency)
  - Aims for terms to have higher specificity, meaning the fewer texts in the dataset contain the term, the better.

Sentence Embeddings: TF-IDF

日記 A (小明的)：「我 今天 去 吃 漢堡，漢堡 超 好吃。」
日記 B (小華的)：「我 今天 去 學校 上課，好累。」
日記 C (小美的)：「我 今天 跟 媽媽 去 買 菜。」

字典大小

	我	今天	去	吃	漢堡	超	好吃	學校	好累	跟	...
TF	1/8 =0.125	1/8 =0.125	1/8 =0.125	1/8 =0.125	2/8 =0.25	1/8 =0.125	1/8 =0.125	0	0	0	
IDF	log(3/3) =0	log(3/3) =0	log(3/3) =0	log(3/1) =0.48	log(3/1) =0.48	log(3/1) =0.48	log(3/1) =0.48	log(3/1) =0.48	log(3/1) =0.48	log(3/1) =0.48	
分數 (TFxIDF)	0	0	0	0.06	0.12	0.06	0.06	0	0	0	

TF-IDF 的問題

1. 在 TF-IDF 的世界裡，字跟字是完全獨立的。
   - 句子 A：「我很**快樂**」
   - 句子 B：「我很**開心**」
2. 本質上仍是詞袋模型 (Bag-of-words) ，不分「順序」
   - 句子 A：「小明喜歡小華」
   - 句子 B：「小華喜歡小明」

Outline

- 文字向量基礎
  - **Introduction to embeddings**
  - Sentence embeddings
  - Word embeddings

Word Representations

Basic Approach

Corpus

I like to eat apple

You like to watch TV

Vocabulary

apple	[1, 0, 0, 0, 0, 0, 0, 0]
eat	[0, 1, 0, 0, 0, 0, 0, 0]
I	[0, 0, 1, 0, 0, 0, 0, 0]
like	[0, 0, 0, 1, 0, 0, 0, 0]
watch	[0, 0, 0, 0, 1, 0, 0, 0]
to	[0, 0, 0, 0, 0, 1, 0, 0]
⋮	

One hot encodings with each vector size equal to the vocab size (8 in this case)

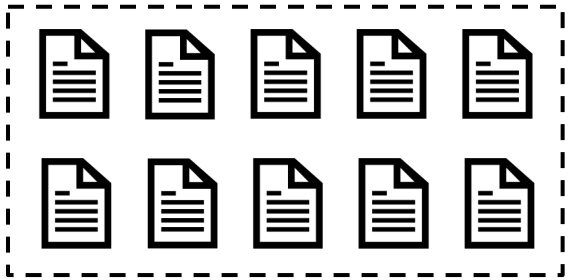
Pros: handy, training-free

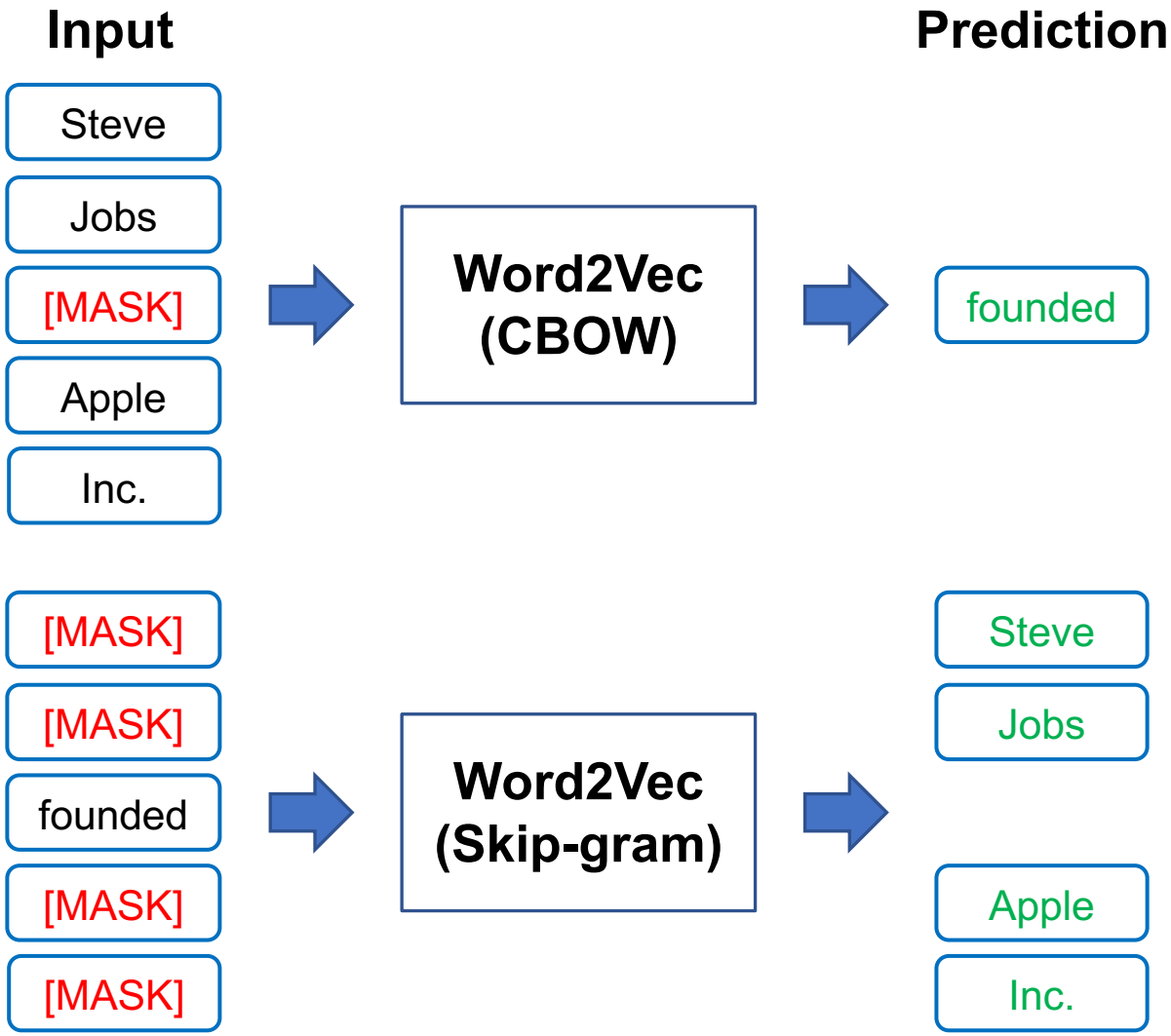
Cons: sparse matrix which occupies much memory

詞嵌入：把詞嵌入到向量 (Word2Vec)

[1] Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

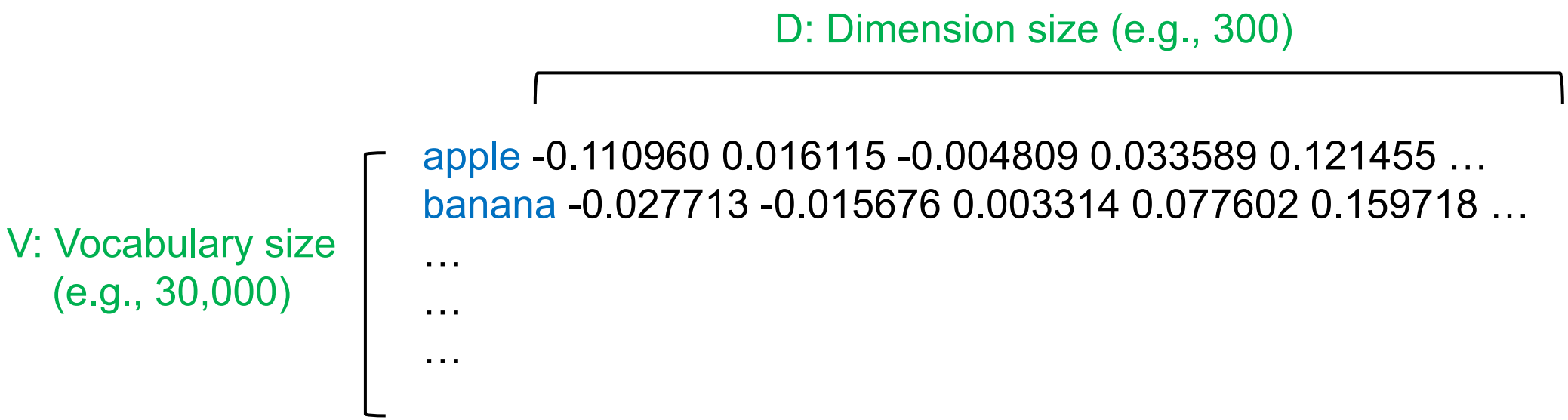
非常大量的語料庫
(例如：Wikipedia)



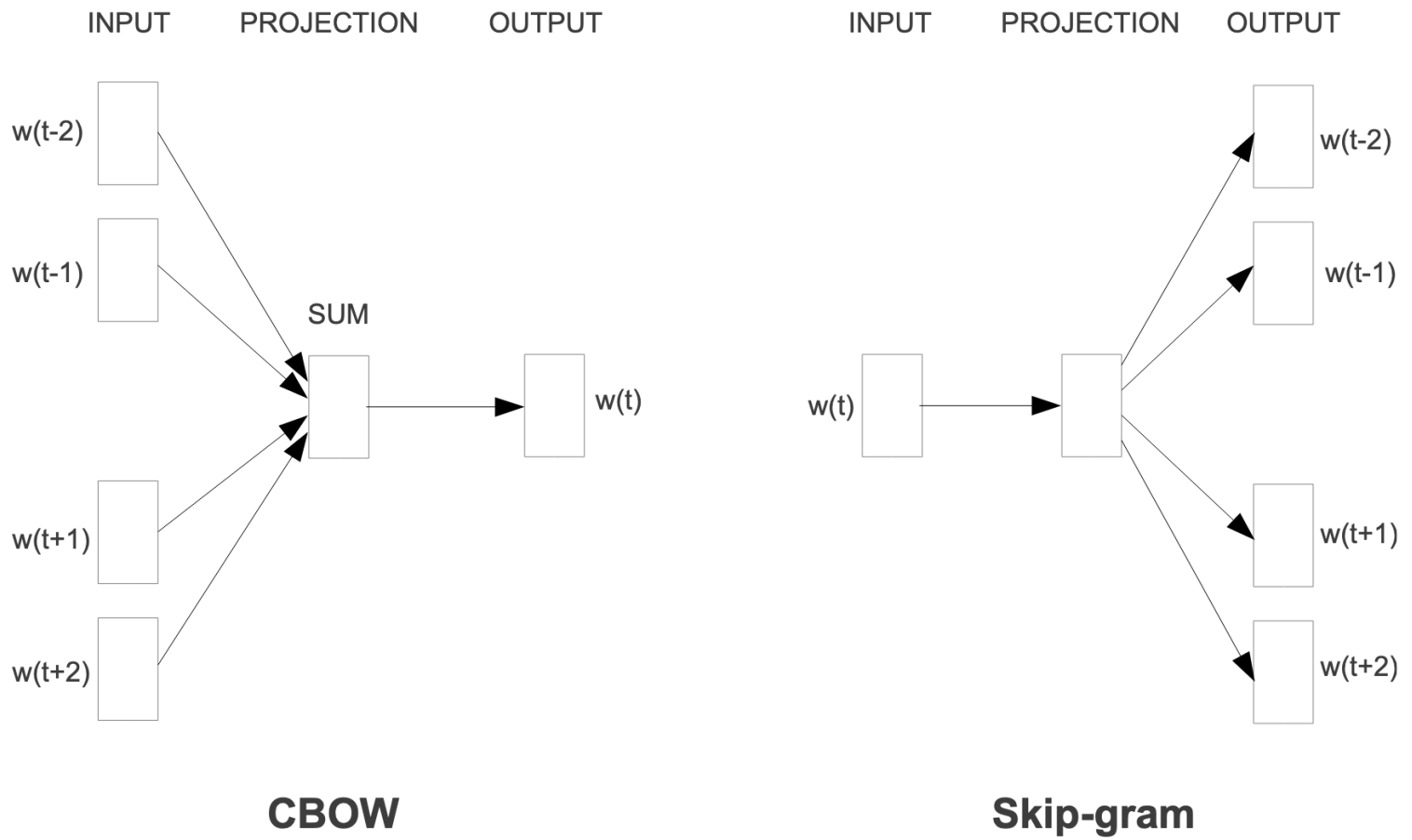


每個字，都是一段向量 – 詞向量

- Word2Vec 的範例長相:

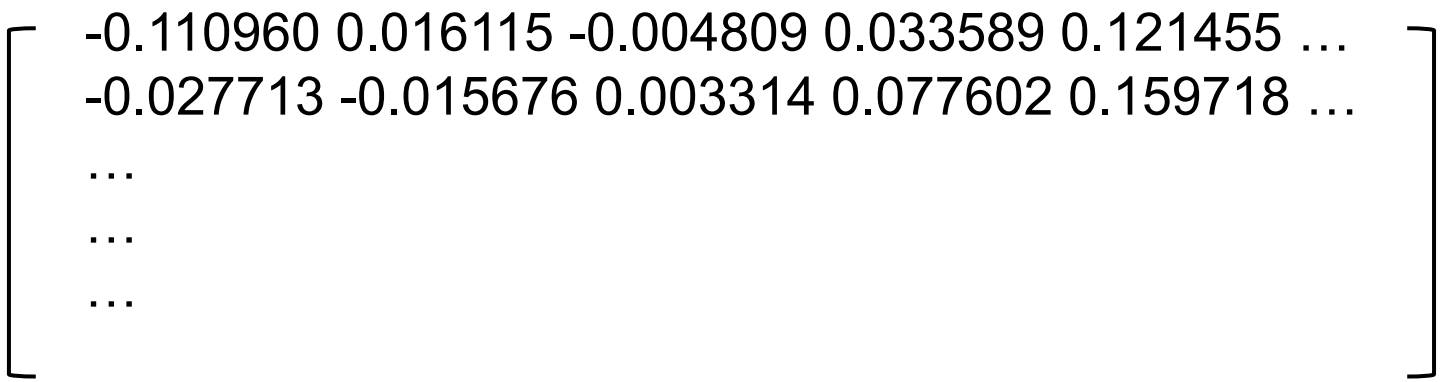


Word2Vec: CBOW vs. Skip-gram



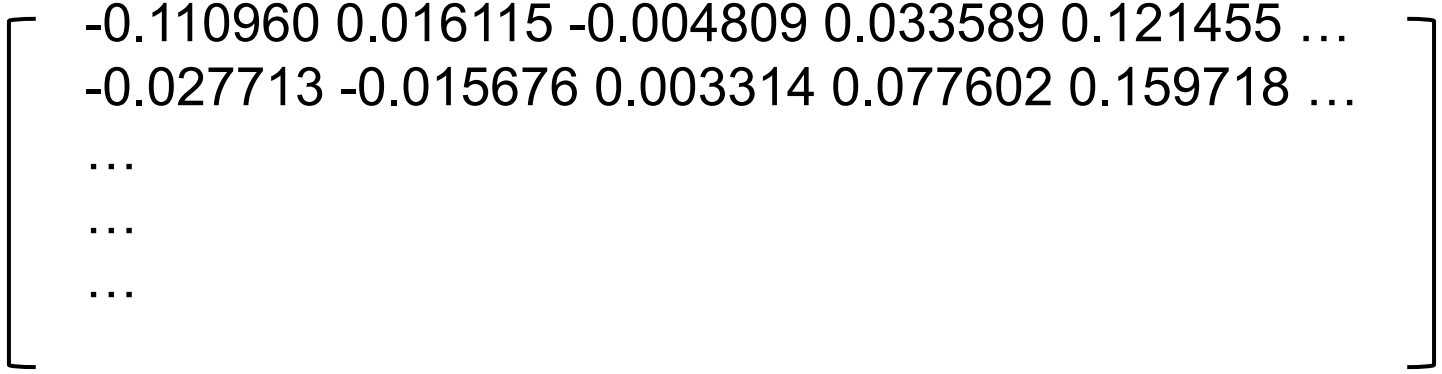
Mikolov et al. "Efficient estimation of word representations in vector space." arXiv preprint arXiv:1301.3781 (2013).

How to train? (以 Skip-gram 為例, 1/2)



W_1: VxD

Input Embeddings



W_2: VxD

Output Embeddings

Embedding Lookup

Index	Vocab	Dimension 1	Dimension 2
0	I	-0.4703061747695665 4	-4.440892098500626e-16
1	love	-8.326672684688674e-16	0.4464710977801294
2	do	8.881784197001252e-16	0.7838949952895327
3	deep	-0.4915010045415975	-1.3016463110087907e-16
4	learning	-1.582067810090848e-15	0.4314767609119055
5	NLP	-0.2523320838384413	-2.1837832516159107e-16
6	programming	-0.6881623238553303	-1.9142202344347 39e-16

How to train? (以 Skip-gram 為例, 2/2)

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\leq j\leq c, j\neq 0}\log p(w_{t+j}|w_t)$$

中間字預測周圍字的機率值

Symbol	Meaning
T	一個序列 $(w_1, w_2, \ldots, w_T)$ 有T個字
c	Context window 的範圍，以前一頁的例子來說：$c = 4$
w_t	中間的字 (代表 $t + j = 0$)

Softmax function

$$p(w_O|w_I) = \frac{\exp\left({v'_{w_O}}^{\top} v_{w_I}\right)}{\sum_{w=1}^{W}\exp\left({v'_w}^{\top} v_{w_I}\right)}$$

內積

Symbol	Meaning
v'_{w_O}	輸出字的向量 (output word vector)
v_{w_I}	輸入字的向量 (Input word vector)
w_O	輸出字 (output word)
w_I	輸入字 (input word)
W	字典大小

為什麼要取 log?

考慮到一些極端數值：

$$10^4 - 10^3$$

如果取log之後：

$$\log 10^4 - \log 10^3 = 4 - 3$$

- 會讓數值範圍變小，使機器學習的最佳化過程穩定
- 會降低極端值的影響，避免 outliers 影響訓練結果

https://datascience.stackexchange.com/questions/40089/what-is-the-reason-behind-taking-log-transformation-of-few-continuous-variables

Softmax

- When generating the next word, `softmax` is performed to get the probabilities among the words in the vocabulary.
- Softmax formula:

$$\frac{\exp(u_l)}{\sum_{l'}^{|V|} \exp\left(u_{l'}\right)}$$

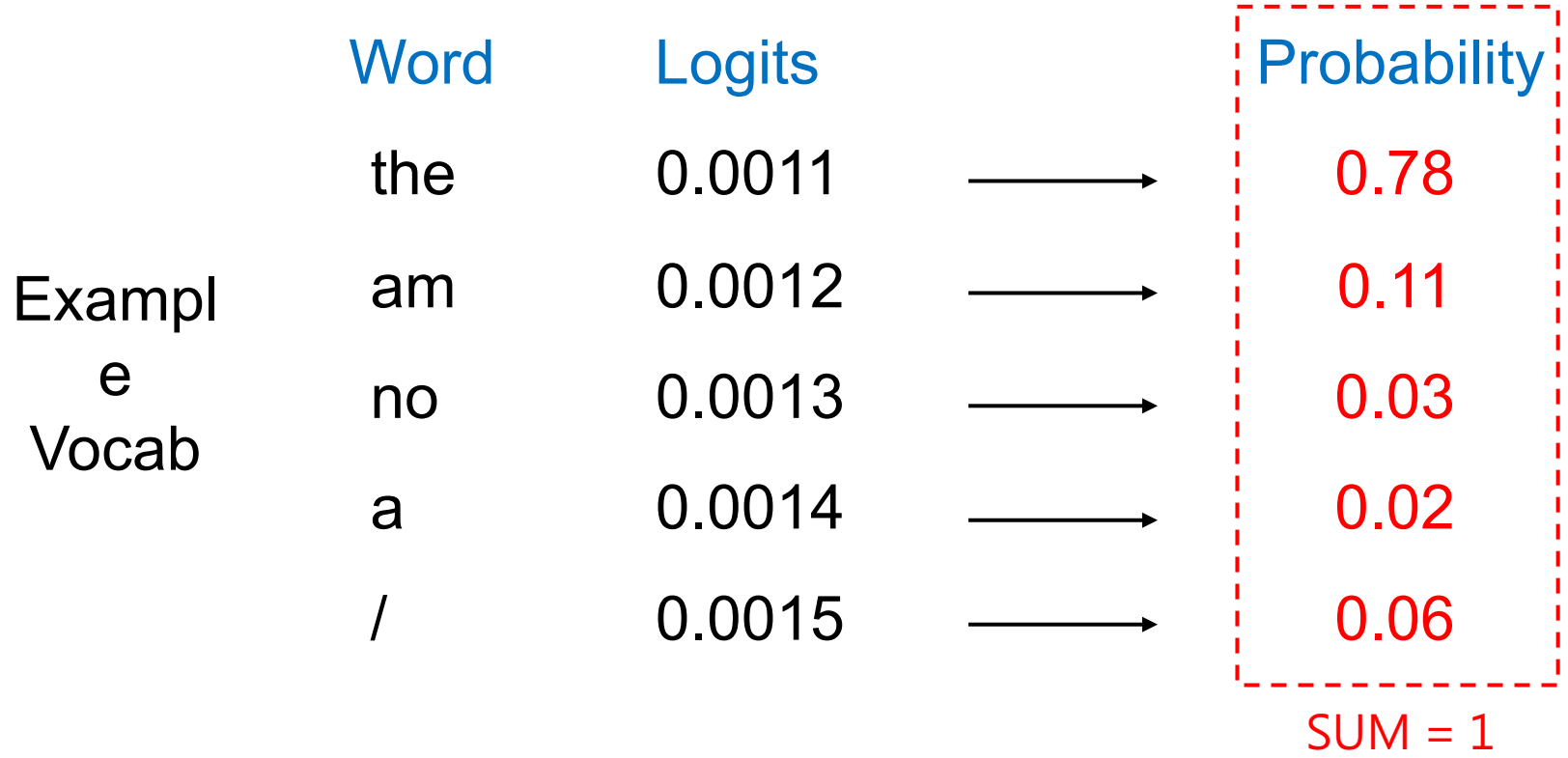
使用 exp 來取得機率的主要原因：

(1) 恆正

(2) 符號不變性

u_l: logits (model outputs before softmax)
$|V|$: size of the vocabulary

Softmax



- Note that softmax is required for every decoding strategies since we need to find out the next word from a vocabulary.

Advanced Techniques of Word2vec

- Speed up the training process of Word2vec
- Hierarchical Softmax
- Negative Sampling

Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

Hierarchical Softmax Overview

- 總目標：近似輸出層，因為在 Vocab size 很大的情況下做 Softmax 計算量很大
- 訓練流程：
  - Step1: 把字典的字根據頻率建立出一棵 Huffman Tree
  - Step2: 以 internal nodes 的權重 (vectors) 代替輸出層，從 root node 開始計算往下走每層的機率值 Softmax(dot_product)，每層都只做二元分類
  - Step3: 調整 internal nodes 的權重

Huffman Tree 是一種最佳前綴編碼樹 (optimal prefix code tree)，其特性是：

- 頻率較高的詞會被分配較短的路徑，使得它們的預測成本較低。
- 頻率較低的詞會有較長的路徑，以減少總體的編碼長度。

Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

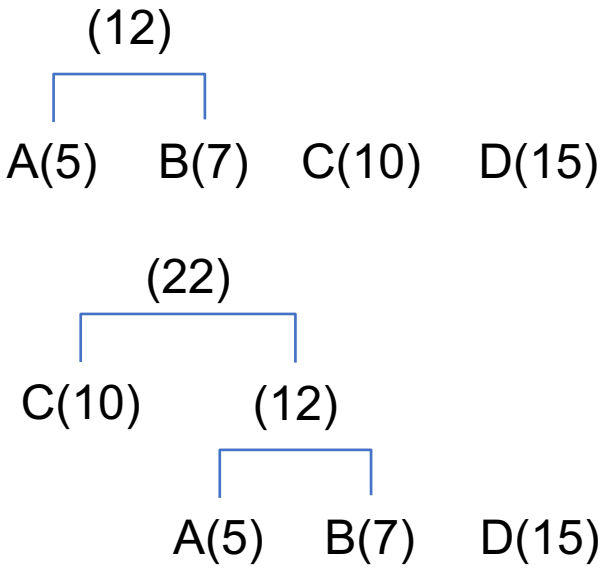
Huffman Tree 快速範例

假設有 4 個符號，出現頻率如下：A: 5、B: 7、C: 10、D: 15

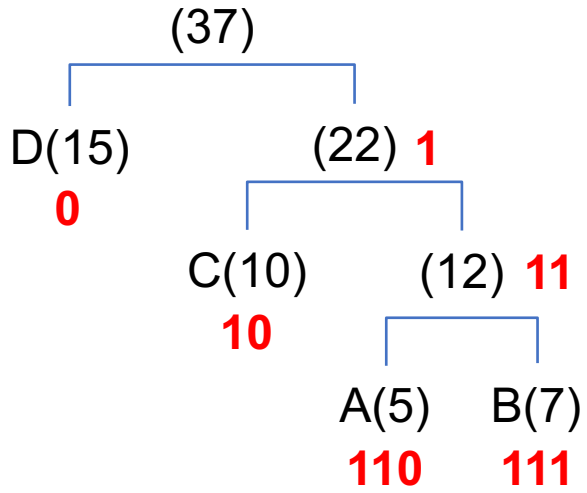
Step 1. 每個符號先當一棵樹

A(5) B(7) C(10) D(15)

Step 2. 每次合併「頻率最小的兩棵樹」

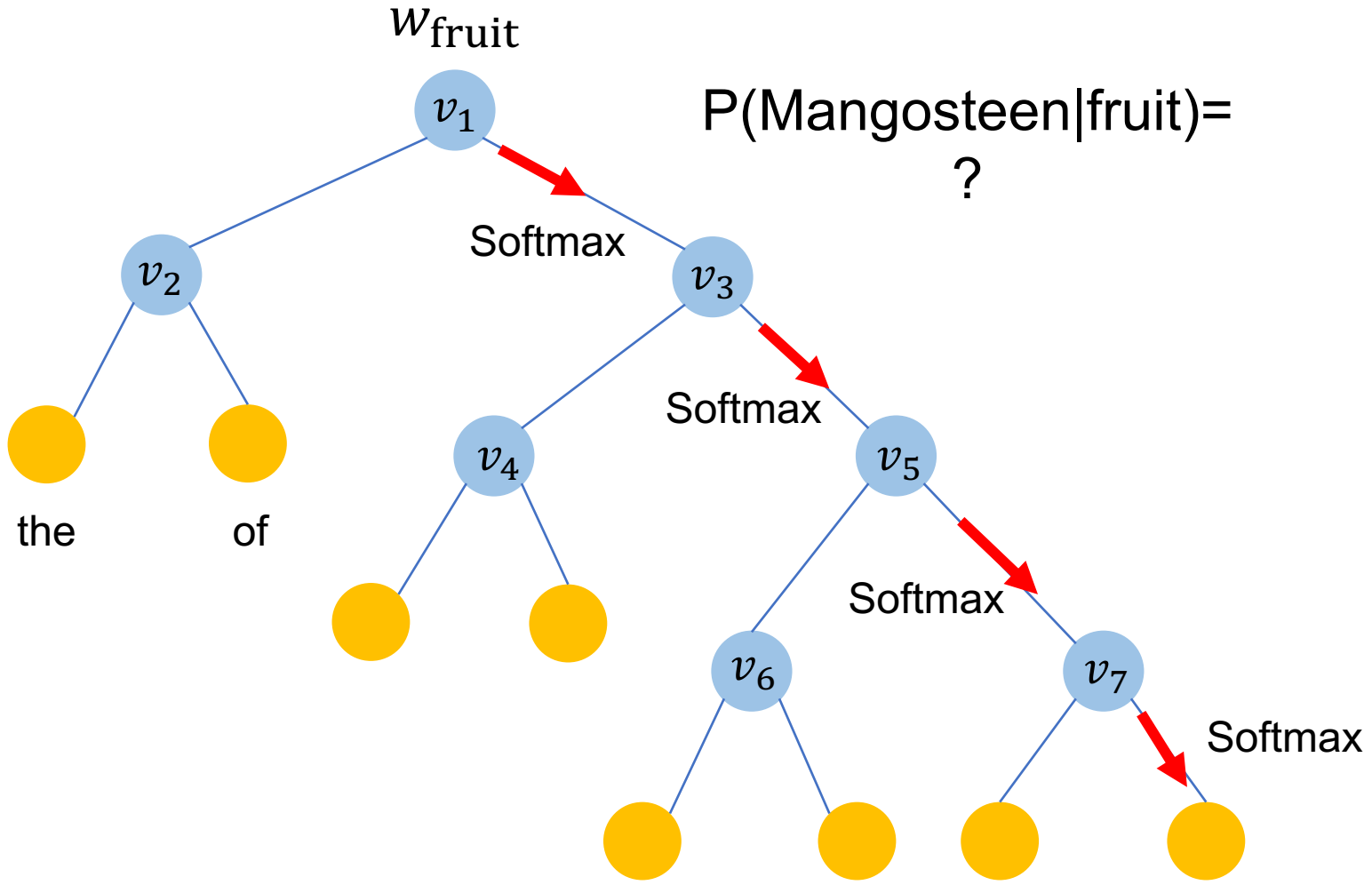


Step 3. 給予標記



Hierarchical Softmax

Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.



P(Mangosteen|fruit)= ?

Internal nodes (帶有 weights)

字典中的字 (越上層詞頻越高，結構為 Huffman Tree)

訓練目標為最大化路徑中每個節點經過Softmax後結果相乘的機率值

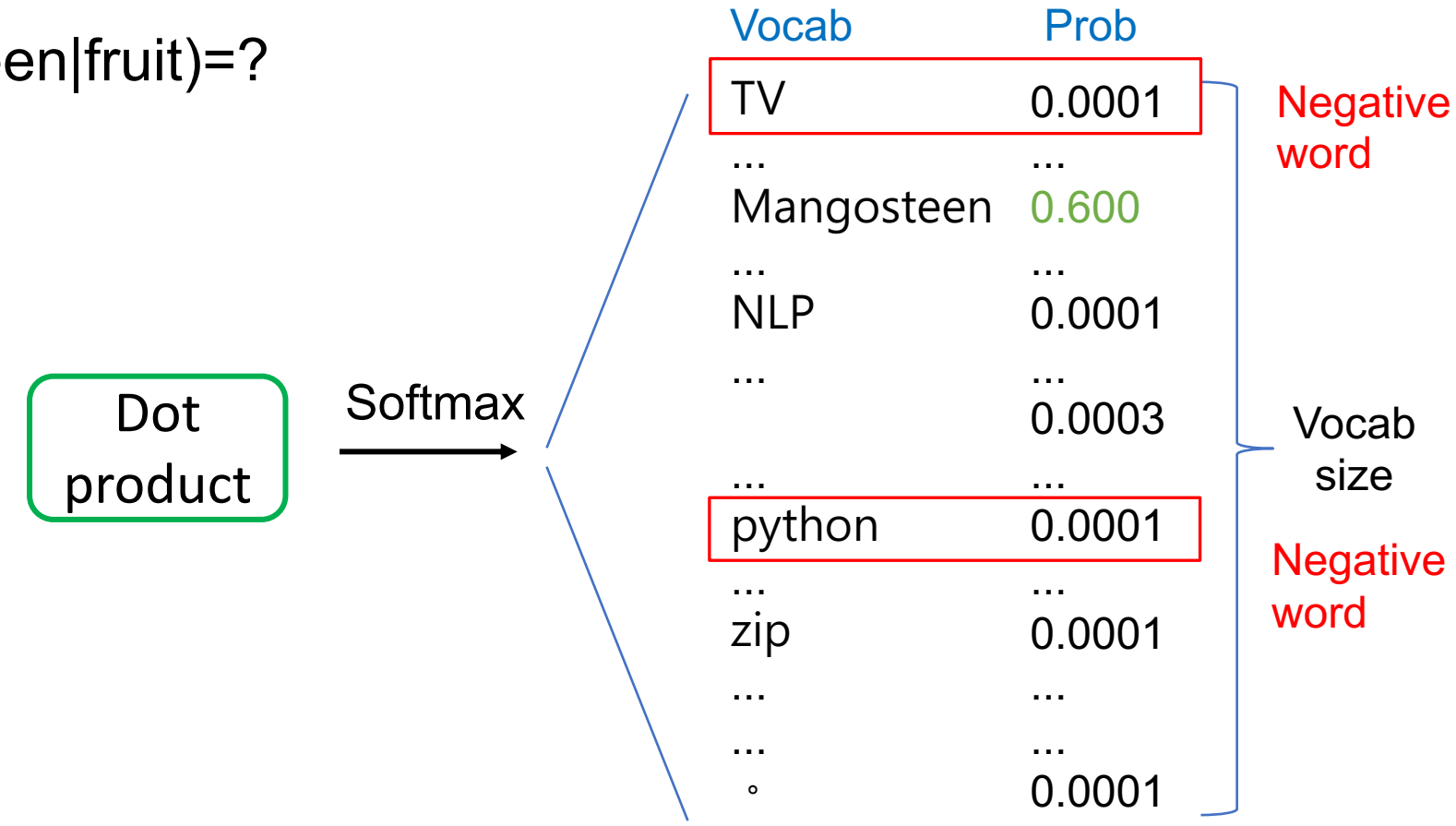
Negative Sampling Overview

- 目標：訓練時不更新整個 $W_{h \to V}$ (將隱藏狀態轉移至字典大小的分類層)，僅更新正確答案以及採樣到 (sampled) 的負向字
- P(Mangosteen|fruit)=?
  - TV 可能跟 fruit 無關
  - python 可能也跟 fruit 無關
- 如何採樣負向的字？論文中使用 Uniform Distribution

Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

Negative Sampling

P(Mangosteen|fruit)=?



Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

Negative Sampling Details

- Negative words 要設定成多少呢？
  - 5–20 are useful for small training datasets
  - 2–5 for large training datasets
  - (Empirical setting; Hyperparameter)

https://radimrehurek.com/gensim/models/word2vec.html

Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

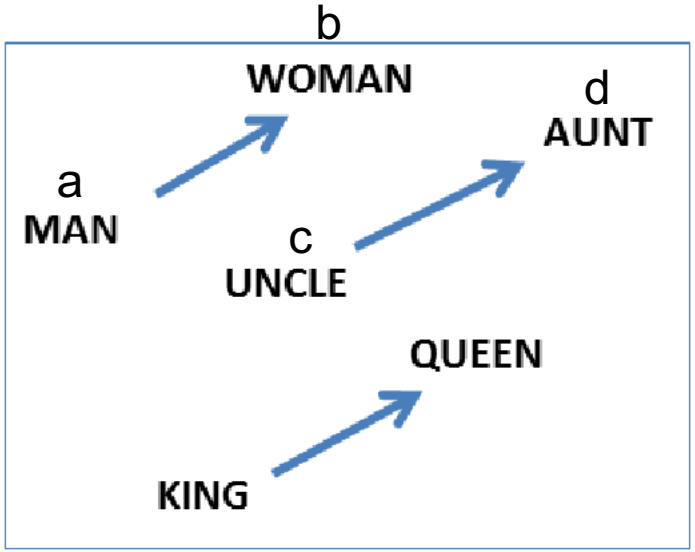
詞向量視覺化

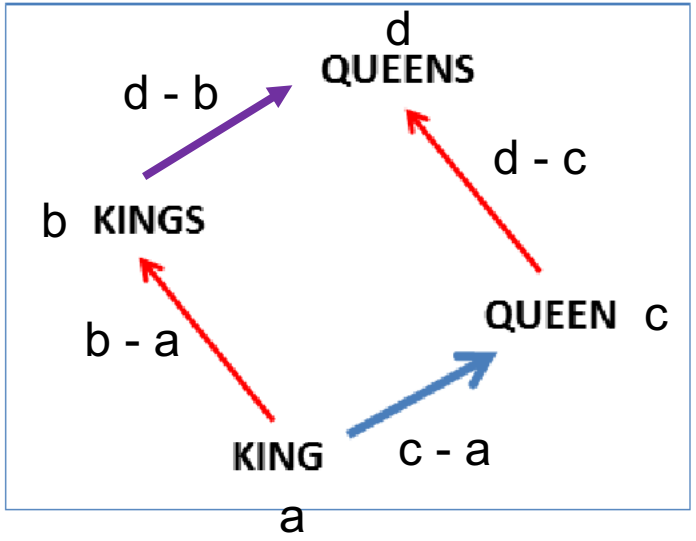
- **Word embedding model**: glove-wiki-gigaword-100
- **Dimension reduction**: t-SNE
- **Dataset**: Mikolov et al., 2013

Try it:
https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/

Word Analogy (左：semantic, 右: syntactic)

b - a = d - c
d = b - a + c





b – a = d – c
(移項) d – b = c - a

Mikolov et al. “Linguistic regularities in continuous space word representations.” NAACL 2013.

The Two Word2vec Papers

- Mikolov et al. "Efficient estimation of word representations in vector space." arXiv preprint arXiv:1301.3781 (2013).
  - 介紹 CBOW 以及 Skip-gram
- Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.
  - 針對 Skip-gram 來進行詳細說明，並且介紹加速訓練的方法：
    - (1) Hierarchical Softmax (2) Negative Sampling

Thank you!

Instructor: 林英嘉

✉ yjlin@cgu.edu.tw

TA: 林宣辰

✉ b1128015@cgu.edu.tw